Second Lab Assignment: System Modeling and Profiling

STUDENTS IDENTIFICATION:

Number:	Name:

2 First steps

Please justify all your answers with values from the experiments.

1. What is the cache capacity of the computer you used (please write the workstation name)?

Array Size	16 KB	32 KB	64KB	128KB	256KB	512KB
t2-t1	0.003671	0.006822	0.017285	0.041184	0.084879	0.209042
# accesses a[i]	1638400	3276800	6553600	13107200	26214400	52428800
# mean access time	2.24×10^{-9}	2.08×10^{-9}	0.264×10^{-9}	3.14×10^{-9}	3.24×10^{-9}	3.99×10^{-9}

O computador usado foi do Lab 6. A cache tem 32KB de tamanho, pois entre as arrays de 32 KB a 64KB houve um salto significativo entre os tempos de acesso (50% a mais que o esperado).

Consider the data presented in Figure 1. Answer the following questions (2, 3, 4) about the machine used to generate that data.

2. What is the cache capacity?

64KB, pois há um salto exponencial para os strides de tamanhos de arrays superiores a 64KB.

3. What is the size of each cache block?

16 B, pois os tempos de leitura/escrita param de aumentar exponencialmente a strides desse tamanho, começando a ficar constantes (misses constantemente)

4. What is the L1 cache miss penalty time?

$t_{miss} - t_{hit} = t_{penalty}$

t_{miss} é aproximadamente 1000ns e t_{hit} é ~400ns, logo $t_{penalty} \hat{=} 600ns$

3 Assignment

3.1.1 Modeling the L1 Data Cache

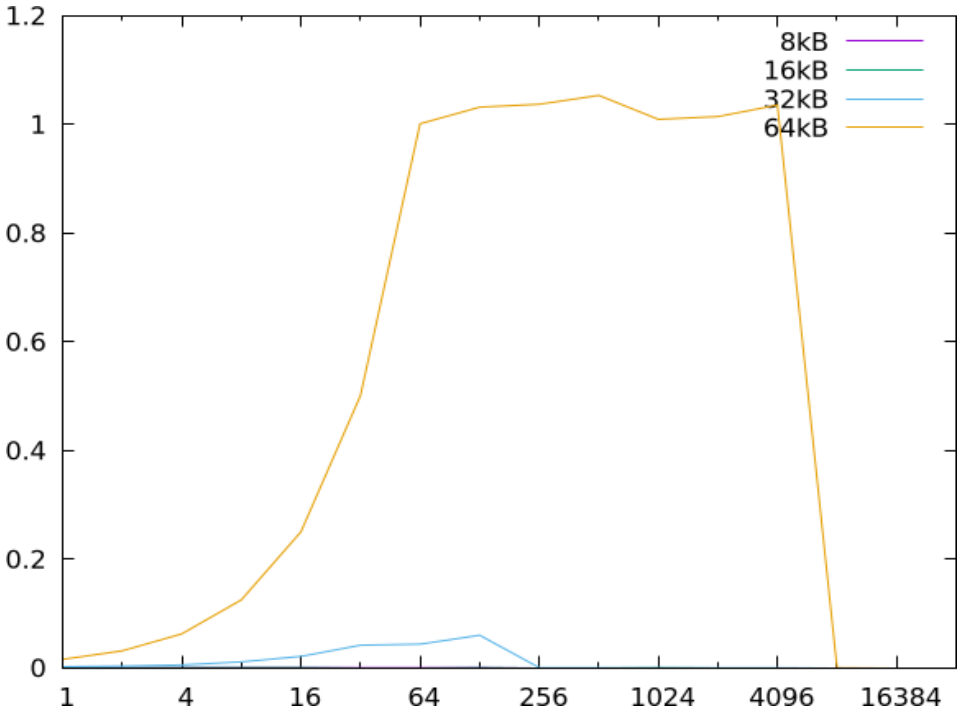
a) What are the processor events that will be analyzed during its execution? Explain their meaning.

Será analisado o número de misses da cache L1 durante a execução do programa, ou seja, a quantidade de vezes que o hardware não encontrou os dados em procura na cache L1 e prosseguiu a procura no próximo nível de cache (L2 ou L3) ou então ao disco, quando não se encontra em cache

b) Plot the variation of the average number of misses (*Avg Misses*) with the `stride` size, for each considered dimension of the L1 data cache (8kB, 16kB, 32kB and 64kB).
Note that, you may fill these tables and graphics (as well as the following ones in this report) on your computer and submit the printed version.

Array Size	Stride	Avg Misses	Avg Cycl Time
8kBytes	1	0.000251	0.002575
	2	0.000168	0.002780
	4	0.000059	0.002774
	8	0.000062	0.002721
	16	0.000049	0.002773
	32	0.000067	0.002762
	64	0.000055	0.002655
	128	0.000040	0.002303
	256	0.000072	0.002062
	512	0.000008	0.002004
	1024	0.000005	0.001967
	2048	0.000004	0.002017
	4096	0.000004	0.002028
16kBytes	1	0.000222	0.002193
	2	0.000177	0.002217
	4	0.000179	0.002186
	8	0.000185	0.002124
	16	0.000184	0.002198
	32	0.000190	0.002160
	64	0.000173	0.002206
	128	0.000120	0.002201
	256	0.000045	0.002072
	512	0.000078	0.001948
	1024	0.000008	0.001956
	2048	0.000007	0.002084
	4096	0.000005	0.002188
	8192	0.000003	0.002043

Array Size	Stride	Avg Misses	Avg Cycl Time
32kBytes	1	0.002197	0.002765
	2	0.003374	0.002157
	4	0.005166	0.002151
	8	0.008571	0.002107
	16	0.015802	0.002020
	32	0.025452	0.001988
	64	0.032005	0.002228
	128	0.054315	0.002240
	256	0.100096	0.002247
	512	0.000150	0.002079
	1024	0.000070	0.001963
	2048	0.000029	0.002073
	4096	0.000018	0.002205
	8192	0.000002	0.002197
	16384	0.000001	0.002040
64kBytes	1	0.015653	0.001956
	2	0.031311	0.001877
	4	0.061691	0.002185
	8	0.125289	0.002212
	16	0.250498	0.002258
	32	0.501097	0.002028
	64	1.000655	0.001877
	128	1.014417	0.001979
	256	1.011220	0.001992
	512	1.050476	0.001901
	1024	1.008602	0.001870
	2048	1.022278	0.002253
	4096	1.101973	0.006733
	8192	0.000015	0.002144
	16384	0.000001	0.002125
	32768	0.000001	0.002074



c) By analyzing the obtained results:

- Determine the **size** of the L1 data cache. Justify your answer.

A cache L1 tem 32KB de capacidade, pois pelos valores obtidos podemos observar que para a array de 32KB, e abaixo o número médio de misses mantém-se relativamente baixo para todas as variações de stride, mas a partir da array de 64KB, os valores para as misses começam a crescer exponencialmente, revelando que até 32KB a array cabia na cache inteira, enquanto que com array de 64KB é preciso ir buscar os dados necessários a hierarquia de memórias mais baixas

- Determine the **block size** adopted in this cache. Justify your answer.

O tamanho do bloco é de 64B, pois quando a array excede a cache (64KB), o número médio de misses cresce exponencialmente com fator 2 em relação aos misses do stride anterior até atingir constantemente 1 miss a cada acesso a partir de strides de 64B, indicando que o salto excedeu o tamanho do bloco a este ponto e que o hardware fica constantemente a ir procurar os dados a memórias mais baixas.

- Characterize the **associativity set size** adopted in this cache. Justify your answer.

A partir de strides de 8KB, o miss rate diminui consideravelmente. Assim, podemos concluir que 4KB é o stride onde os dados necessários são sempre mapeados ao mesmo set: no caso da array de 64KB, 64/4 = 16 blocos distintos em um set que resulta em misses constantes; quando passamos a strides de 8KB: 64/8 = 8 blocos distintos mapeados sempre no mesmo set com misses muito próximos de 0, ou seja, os 8 blocos cabem todos num set. Logo, cada set contém 8 blocos (8-way associative, com 512 B de tamanho para cada set).

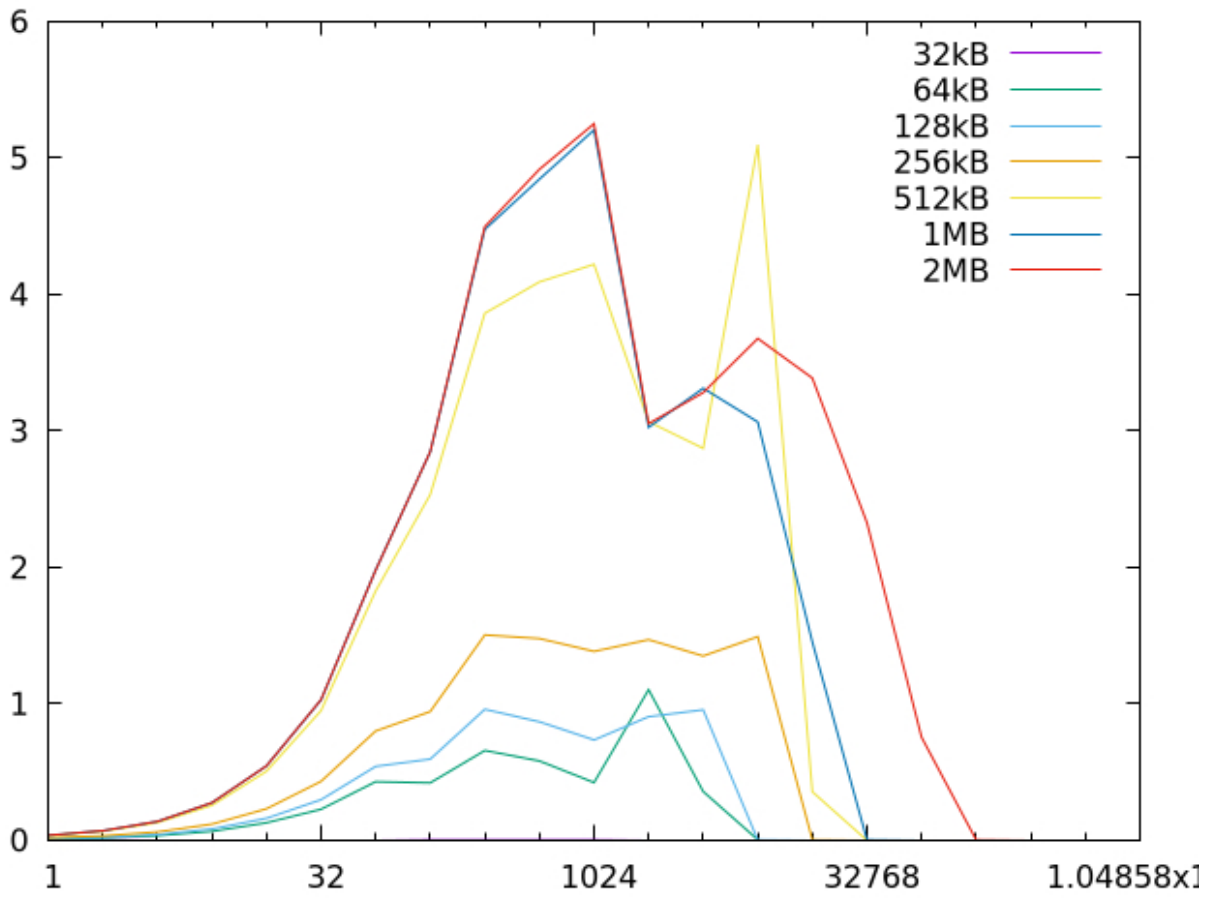
3.1.2 Modeling the L2 Cache

a) Describe and justify the changes introduced in this program.

Mudificamos o evento em análise para contar o número de misses da cache L2 (PAPI-L2_DCM) e aumentamos o tamanho das arrays que iam ser usadas. Uma vez que o objetivo é analisar a cache L2, as arrays passam a ser de 32KB (tamanho da cache L1) a 2MB, forçando o programa a exceder a cache L1 e fazer uso do L2.

b) Plot the variation of the average number of misses (*Avg Misses*) with the `stride` size, for each considered dimension of the L2 cache.

s



c) By analyzing the obtained results:

- Determine the **size** of the L2 cache. Justify your answer.

O tamanho é de 256KB, pois há uma grande distinção entre o número de misses de tamanhos de array inferiores e os superiores, nos quais os superiores têm a média de misses a crescer exponencialmente

- Determine the **block size** adopted in this cache. Justify your answer.

Pelo gráfico pode-se observar que existem misses constantemente a partir de strides de 32B, ou seja, o stride passou do limite de um bloco a cada salto. Logo, o tamanho de cada bloco é 32B

- Characterize the **associativity set size** adopted in this cache. Justify your answer.

Para arrays maiores que o tamanho da cache, é observável que têm média de 0 misses pela primeira vez em strides de 32kB, stride este que é o suficiente para saltar o número de sets totais e mapear os dados sempre no mesmo set. Assim, a associatividade da cache é

$\frac{\text{cache size}}{\text{stride}} = \frac{256\,kB}{32\,kB} = 8$ way - set associativity.

3.2 Profiling and Optimizing Data Cache Accesses

3.2.1 Straightforward implementation

a) What is the total amount of memory that is required to accommodate each of these matrices?

Tamanho da matriz x Tamanho dos elementos = $N^2 \times 2B = 512^2 \times 2B =$
$= 512kB$

Cada matriz ocupa 512 kB. Como o código usa três matrizes do mesmo tamanho, será usado no total 1.5MB de memória.

b) Fill the following table with the obtained data.

Total number of L1 data cache misses	135.023447 $\times 10^6$
Total number of load / store instructions completed	536.871914 $\times 10^6$
Total number of clock cycles	594.244552 $\times 10^6$
Elapsed time	0.198082 seconds

c) Evaluate the resulting L1 data cache *Hit-Rate*:

$$\frac{536.871914 - 135.023447}{536.871914} \times 100 \simeq 74.57\,\%$$

L1 Hit-Rate = 74.57 %

3.2.2 First Optimization: Matrix transpose before multiplication [2]

a) Fill the following table with the obtained data.

Total number of L1 data cache misses	4.220054 $\times 10^6$
Total number of load / store instructions completed	536.871898 $\times 10^6$
Total number of clock cycles	533.895403 $\times 10^6$
Elapsed time	0.177965 seconds

b) Evaluate the resulting L1 data cache *Hit-Rate*:

$$\frac{536.871898 - 4.220054}{536.871898} \times 100 \simeq 99.21\%$$

L1 Hit-Rate = 99.21%

c) Fill the following table with the obtained data.

Total number of L1 data cache misses	4.482979 $\times 10^6$
Total number of load / store instructions completed	537.396788 $\times 10^6$
Total number of clock cycles	531.573828 $\times 10^6$
Elapsed time	0.177192 seconds

Comment on the obtained results when including the matrix transposition in the execution time:

Pelos dados, pode-se ver que o tempo que leva para transpor uma matriz é muito baixo e até basicamente insignificante comparado ao tempo gasto na multiplicação

d) Compare the obtained results with those that were obtained for the straightforward implementation, by calculating the difference of the resulting hit-rates (ΔHitRate) and the obtained speedups.

$\Delta HitRate = HitRate_{mm2} - HitRate_{mm1}$: 24.64 %
$Speedup(\#Clocks) = \#Clocks_{mm1}/\#Clocks_{mm2}$: 1.113
$Speedup(Time) = Time_{mm1}/Time_{mm2}$: 1.113
Comment: Observa-se que ao transpor a matriz leva a um aumento de velocidade em 11%, devido também ao aumento do hit-rate. Pode-se concluir que o tempo perdido na transposição é facilmente recuperado pelo alto hit-rate; e que a forma como um programa acessa a memória pode influenciar nos seus tempos de execução; melhorando o acesso à localidade dos dados, neste caso

3.2.3 Second Optimization: Blocked (tiled) matrix multiply [2]

a) How many matrix elements can be accommodated in each cache line?

Com um cache block de 64B, será possível acomodar 32 elementos da matriz (2B cada).

b) Fill the following table with the obtained data.

Total number of L1 data cache misses	3.761307 $\times 10^6$
Total number of load / store instructions completed	537.547791 $\times 10^6$
Total number of clock cycles	230.182973 $\times 10^6$
Elapsed time	0.076727 seconds

c) Evaluate the resulting L1 data cache *Hit-Rate*:

$$\frac{537.547791 - 3.761307}{537.547791} \times 100 \simeq 99.3\%$$

L1 Hit-Rate = 99.3%

d) Compare the obtained results with those that were obtained for the straightforward implementation, by calculating the difference of the resulting hit-rates (ΔHitRate) and the obtained speedup.

ΔHitRate = $\text{HitRate}_{mm3} - \text{HitRate}_{mm1}$: 24.73%
Speedup(#Clocks) = $\#\text{Clocks}_{mm1}/\#\text{Clocks}_{mm3}$: 2.582
Comment: Observa-se que um alto hit-rate resulta em tempos de execução mais baixos. Novamente, a melhoria no acesso à localidade dos dados resultou num hit-rate mais alto

e) Compare the obtained results with those that were obtained for the matrix transpose implementation by calculating the difference of the resulting hit-rates (ΔHitRate) and the obtained speedup. If the obtained speedup is positive, but the difference of the resulting hit-rates is negative, how do you explain the performance improvement? (Hint: study the hit-rates of the L2 cache for both implementations;)

ΔHitRate = $HitRate_{mm3} - HitRate_{mm2}$: 0.09%
Speedup(#Clocks) = $\#Clocks_{mm2}/\#Clocks_{mm3}$: 2.379
Comment: Através do PAPI, verificou-se que o número de acessos à cache L2 para as duas formas de cálculo são basicamente iguais. Porém, o número de misses na L2 usando a matriz transposta foi 4 vezes superior aos misses obtidos usando o cálculo por blocos, explicando o facto do tempo de processo ser tão diferente, mesmo o hit-rate na L1 ser praticamente igual para as duas formas.

3.2.3 Comparing results against the CPU specifications

Now that you have characterized the cache on your lab computer, you are going to compare it against the manufacturer's specification. For this you can check the device's datasheet, or make use of the command `lscpu`. Comment on the results.

Para obter os resultados reais utilizamos o comando lscpu: Para a cache L1/L2 observamos um total de 32KB/256KB que se verifica com o tamanho real das caches